

MODULES - Technical Decisions and Concepts

This file explains what each core technology in the system does, why it was chosen over the alternatives, and where to find it in the codebase.

Remote Photoplethysmography (rPPG)

File: `micro_expressions/rppg_heart_rate.py`

Library: `vitallens` (v0.6+) - <https://github.com/Rouast-Labs/vitallens-python>

What it is

rPPG is a technique for estimating heart rate from ordinary video. Blood pulsing through skin vessels causes tiny periodic changes in skin colour, particularly in the green channel. The human eye cannot see it. A camera can.

The system uses the VitalLens library's real-time streaming API. Each frame, a face region is pushed to VitalLens which runs the POS algorithm internally and returns rolling BPM estimates and a PPG waveform. HRV (RMSSD) is computed from the peaks in that waveform.

If VitalLens is not installed, the module falls back to a green-channel FFT method using `scipy`.

How VitalLens is integrated

VitalLens is initialised with `Method.POS` (local processing, no API key needed) and `detect_faces=False` (we pass the face box from our existing MediaPipe landmarks). A streaming session is opened on first frame via `vl.stream()`, and each frame is pushed with `session.push(rgb_frame, timestamp, face=box)`. Results are pulled non-blocking with `session.get_result(block=False)`.

The face bounding box is derived from the same ROI landmark indices used previously, padded by 30% on each side.

Algorithm used: POS (Plane-Orthogonal-to-Skin)

POS projects the RGB skin signal onto a plane orthogonal to the current skin-tone vector. This separates the blood volume pulse from motion artifacts and lighting changes. The skin-tone vector is recomputed each window, adapting to different skin tones and varying illumination.

Wang et al. 2017 - "Algorithmic Principles of Remote PPG".

Why VitalLens over other rPPG libraries

VitalLens supports POS, CHROM, and GREEN locally without an API key. It has a proper streaming API designed for real-time webcam use, handles signal windowing and buffering internally, and is actively maintained.

Alternative	Problem
yarPPG	Lightweight but only supports GREEN channel, no POS/CHROM
open-rppg	Deep learning models, requires JAX (heavy dependency)
pyVHR	Research framework, last updated Jan 2023, batch-oriented, requires PyTorch
bob.rppg.base	Abandoned, no longer maintained
Manual POS implementation	Works but no streaming API, no built-in signal quality handling

Why POS over CHROM or GREEN

Alternative	Problem
CHROM (Chrominance-based)	Better in ideal conditions but degrades faster under partial occlusion or shadows
GREEN channel only	Simple but picks up breathing artefacts and lighting flicker
Contact PPG (pulse oximeter)	Requires the player to wear a sensor; not practical at a poker table

What degrades the signal

- Head movement blurs the skin region across skin and non-skin pixels
- Fluorescent lights flicker at mains frequency (50-60 Hz) which aliases into the signal
- Highly reflective skin or heavy make-up reduces the absorption contrast

The system tracks HR variance across the hand buffer and disables HR-based updates for that hand if variance exceeds `NOISY_HR_VAR_THRESHOLD` (default: 100.0). This is configurable in `config.py`.

HRV (Heart Rate Variability)

RMSSD is computed from the inter-beat intervals detected in the PPG waveform returned by VitalLens. HRV drops faster than raw BPM when stress hits (typically within 2-3 seconds vs 5-10 seconds for BPM), making it a more responsive early indicator of arousal.

Fallback mode

If `vitalLens` is not installed, the module automatically falls back to a green-channel FFT approach using `scipy.signal`. This extracts the mean green channel value from the face ROI, applies a butterworth bandpass filter (0.7-3.0 Hz), and finds the dominant frequency via FFT. Less accurate than POS but has zero additional dependencies.

Face Detection and Landmark Extraction

File: `micro_expressions/face_detection.py`

What it is

MediaPipe FaceLandmarker detects a face and outputs 468 3D landmarks (x, y, z) covering the full face mesh. These landmarks are the input to both the AU measurement system and the face embedder.

Why MediaPipe

Alternative	Problem
dlib 68-point detector	Slower on CPU, fewer landmarks (68 vs 468), no 3D depth estimate
OpenCV Haar cascades	Detection only, no landmarks, inaccurate with non-frontal faces
OpenFace	Accurate AU detection but requires specific build environment, not pip-installable
DeepFace	Focuses on identity and emotion labels, not raw landmark geometry

MediaPipe runs efficiently on CPU without a GPU, produces dense landmarks in real time at 30fps, and is pip-installable without any system dependencies. The `.task` model file is bundled with the repo.

What the 468 landmarks cover

The landmark set covers the full face mesh including eyelids, lip interior, and iris. The system primarily uses landmarks around the brow, eye corners, jaw, and lip corners for AU computation, and a set of 64 pairs distributed across the face for the identity embedding.

Facial Action Coding System (FACS) and Action Units

File: `micro_expressions/facs_action_units.py`

What it is

FACS is a standardised system developed by Paul Ekman for describing facial expressions in terms of individual muscle movements rather than emotion labels. An Action Unit (AU) is a single muscle group or combination, assigned a number by the standard.

The system measures 6 AUs:

AU	Muscle	Expression it indicates
AU1	Inner frontalis	Inner brow raised - worry, sadness
AU2	Outer frontalis	Outer brow raised - surprise
AU4	Corrugator	Brow lowered and pulled in - concentration, anger
AU12	Zygomaticus major	Lip corner pulled up - smile
AU26	Masseter (relaxed)	Jaw drop - relaxation, surprise, talking
AU45	Orbicularis oculi	Eye closure - blink rate

Why geometry-based rather than model-based

AU intensity is computed from landmark distances normalised by face width. For example, AU4 is measured as the distance between the inner brow landmarks relative to the distance at the person's resting state. This requires no pre-trained AU classifier.

The alternative is a trained regressor (like OpenFace uses) that maps raw landmark positions to AU intensities via a neural network. That approach generalises better to subtle expressions but requires a trained model file and is significantly more complex to deploy. For poker tell detection, the most informative AUs (brow tension, jaw relaxation, lip movement during talking) are large enough to detect geometrically.

The stress detector also uses AU1+AU2 together (upper face tension) and AU12+AU26 together (which Claude uses during baseline filtering to identify talking frames).

Stress Score Calculation

File: `micro_expressions/stress_detector.py`

What it is

A weighted z-score combination of physiological deviations from the player's personal baseline. The output is a 0-100 score labelled Calm / Mild / Moderate / High / Extreme by `stress_classifier.py`.

The score is:

```
0.35 * HR_delta
+ 0.25 * AU4_delta      (brow furrow)
+ 0.20 * (AU1+AU2)_delta (brow raise)
+ 0.20 * blink_rate_delta
+ HRV_factor            (scaled contribution from RMSSD drop)
```

All deltas are z-scores against the player's own baseline, so a naturally expressive person is not flagged as stressed just because they raise their brows often.

Why not a pre-trained emotion classifier

Pre-trained emotion models (e.g. FER+, AffectNet-based) output labels like "angry", "fearful", "surprised" based on population averages. A poker player suppressing stress will not look conventionally fearful. The signals that matter are small deviations from that specific person's resting state. A personalised z-score approach captures this where a population model would miss it.

Player Identity: Face Embedding

File: `adaptive_learning/face_embedder.py`

What it is

When a face is detected, the system selects 64 landmark pairs from across the face and measures the distance between each pair (normalised by face width for scale invariance). Each pair contributes an x-distance and y-distance, producing a 128-dimensional vector.

Cosine similarity is computed between this vector and every stored profile. A similarity of 0.85 or above loads the existing profile. Below that threshold, a new profile is created. The threshold is configurable in `config.py` as `FACE_SIMILARITY_THRESHOLD`.

Why not a deep embedding model (FaceNet, ArcFace)

Deep embedding models produce more robust identity vectors across lighting, angle, and expression changes, but they require a large model file (~100MB) and a GPU or meaningful CPU time. For a poker table scenario where the same face appears repeatedly at similar angles and lighting, geometric landmark distances are stable enough and run at essentially zero cost.

Bluff Prediction: Thompson Sampling Contextual Bandit

File: `adaptive_learning/opponent_model.py`

What it is

The bandit maintains a Beta distribution $\text{Beta}(\alpha, \beta)$ for each "context bucket". When a showdown is recorded, α is incremented if the player was bluffing, β if they were not. At prediction time, a probability is drawn from the distribution. The drawn value is the bluff probability shown on screen.

A context bucket combines three factors: the player's personality type (aggressive, tight, unpredictable, standard), their current stress level (5 bands), and their HR state (3 bands). Each combination gets its own independent model.

Why Thompson Sampling rather than a neural network

- **Data efficiency.** A neural network needs many examples before it generalises. The bandit produces useful estimates after 2-3 observations per bucket because it only needs to track alpha and beta.
- **Uncertainty is explicit.** When alpha and beta are both small (few observations), the distribution is wide and draws are noisy. This naturally expresses low confidence. A neural net trained on sparse data would not represent uncertainty this way without significant additional engineering.
- **Per-player.** Each player has their own bandit state. No cross-player generalisation is attempted, which would be wrong (one player's stress tells are not another's).
- **Persistent across sessions.** Alpha and beta are written to SQLite after every showdown and reloaded at recognition. The model does not reset between sessions.

Cold-start priors

When a new player is seen for the first time, Claude generates starting alpha/beta values for each bucket based on the just-collected baseline physiology. This replaces the previous approach of uniform priors (which gave near-random predictions for the first two hands) and a simple 3-rule heuristic.

Claude Integration

Files: `adaptive_learning/claude_advisor.py`, `adaptive_learning/adaptive_learning_system.py`, `adaptive_learning/baseline_extractor.py`, `adaptive_learning/profile_store.py`

Why Claude Code CLI rather than the API

The system uses the Claude Code CLI (`claude --print`) via subprocess rather than the Anthropic API. This means:

- No API key required
- No separate billing
- Uses your existing `claude.ai` subscription
- Works as long as the CLI session is authenticated

The binary path is set in `.env` as `CLAUDE_EXE`. The system also checks `PATH` first, so if `claude` is on your path no `.env` change is needed. The model and timeout are also configurable: `CLAUDE_CLI_MODEL` (default `haiku`) and `CLAUDE_TIMEOUT` (default 45 seconds).

Haiku is the default because the tasks are structured JSON extraction and signal interpretation. They do not need the reasoning depth of a larger model, and Haiku completes them quickly enough that the background thread finishes well before the next relevant event.

How the subprocess call works

`_call_claude_cli` in `claude_advisor.py` runs:

```
claude --print --model haiku --output-format text --no-session-persistence --tools "" --append-system-p
```

Key flags:

- `--print` runs a single non-interactive query and exits
- `--no-session-persistence` prevents Claude from creating a session file for this call
- `--tools ""` disables all tool use so Claude only outputs text
- `--output-format text` suppresses any markdown wrapper around the response

The system prompt tells Claude to return only a valid JSON object with no explanation outside it. The response is parsed with a regex that extracts the first `{ . . }` block, so any surrounding text Claude may add does not break parsing.

How blocking is avoided

None of the four Claude calls happen on the main camera loop thread. Each is wrapped in a `threading.Thread(daemon=True)` and started immediately. The rest of the program continues without waiting.

The showdown case specifically: the showdown is written to the database immediately (with `claude_result=None`), the hand buffer is cleared, and the game resumes. When Claude finishes in the background, `profile_store.update_showdown_claude_result` patches that specific row using its `rowid`. The camera loop never sees a delay.

```
Thumb gesture fires
    |
    v
Showdown logged to DB --> Camera loop continues immediately
claude_result = None
    |
    v (background thread)
Claude CLI called (~3-8s)
    |
    v
DB row patched with Claude result
```

The four integration points

Integration 1 - Baseline quality filtering

Triggered in `adaptive_learning_system._post_calibration_background`, which is launched as a daemon thread the moment calibration finishes.

`claude_advisor.analyze_baseline` receives the raw calibration sample buffer downsampled to at most 150 frames. Each frame contains HR, stress score, and AU12/AU26/AU45/AU1/AU2 values. Claude identifies frames where the player was talking (AU12+AU26 both elevated) or turning their head (AU1+AU2 elevated) and returns the clean frame ranges as index pairs plus a verdict of GOOD, FAIR, or POOR.

`baseline_extractor.refine_from_clean_ranges` then rebuilds the mean and std dev baseline using only those clean frames. The quality metadata is stored in `self.quality_info` and displayed on screen.

Integration 2 - Showdown signal analysis

Triggered in `adaptive_learning_system.on_showdown` after the thumb gesture fires. The full hand buffer (all frames from `start_hand` to the gesture) is passed to `claude_advisor.analyze_hand`.

Claude receives pre-computed statistics from the buffer: peak HR delta, stress delta at peak, AU values at peak, HR variance, how far into the hand the peak occurred, and the last 20 showdown records for that player. It returns a structured JSON verdict including prediction, confidence, `p_bluff`, which AUs dominated, and a one-paragraph reasoning note.

This result is stored in the showdowns table alongside the bandit and personality updates.

Integration 3 - Cold-start priors

Triggered once per new player, inside `_post_calibration_background`, after Integration 1 has completed. It only runs if no bandit state already exists for this player.

`claude_advisor.generate_cold_start_priors` receives the validated baseline: resting HR, baseline stress, blink rate, and AU resting values. Claude returns a dict of 9 bluff probability estimates, one per two-factor context bucket (HH, HM, HL, MH, MM, ML, LH, LM, LL where H/M/L describe stress and HR level). These become the starting alpha values in the bandit via `opponent_model._apply_priors`.

Integration 4 - Session debrief

Triggered in `adaptive_learning_system.on_face_lost`, which is called from `unified_ar_system.py` when the face disappears. It fires once per session after the face has been absent for 60 seconds.

`claude_advisor.generate_session_debrief` receives the full showdown history for the current session, the personality state dict (bluff base rate, stress-bluff slope, HR-bluff slope, consistency), and the bandit's context summary (alpha/beta ratios per bucket). Claude writes a plain-English paragraph in second-person note style describing patterns, which tells were reliable, and what to watch for next session.

The note is saved to `profile_store.save_session_notes` and shown on screen for 5 seconds the next time the player is recognised.

What is sent and what is not

Claude receives pre-computed numeric statistics only. No images, no video frames, no face embeddings, and no personally identifying information are ever sent. Specifically:

- AU intensity values (6 numbers per frame, downsampled to at most 150 frames)
- Heart rate delta and variance
- Stress score delta
- Showdown outcomes (prediction vs actual, context bucket, peak stats, timestamps)
- Personality state numbers (4 scalar values)
- Bandit context summary (alpha/beta ratios per bucket)

Card Detection: YOLOv8

File: poker_hand/poker_engine_unified.py, model: poker_hand/poker_v1.pt

What it is

A YOLOv8 nano model trained on ~24,000 images of all 52 standard playing cards. It runs at `imgsz=640` every other frame (configurable via `YOLO_SKIP` and `YOLO_IMGSZ` in `config.py`).

Why YOLO

Alternative	Problem
Template matching	Fails with rotation, lighting variation, partial occlusion
Classic CV (contour detection + suit/rank crop)	Brittle; requires controlled lighting and card position
ResNet/EfficientNet classifier	Classification only; does not locate multiple cards in a scene

YOLO handles multiple cards at different positions and angles in a single pass. The nano variant runs in real time on CPU at 640px input.

Stability buffer

Raw detections are noisy. The system requires a card to appear in at least `CARD_STABILITY_THRESHOLD` (default: 4) consecutive frames before showing it, and `CARD_FINALIZE_THRESHOLD` (default: 12) frames to lock it in. A locked card persists for up to `CARD_FADE_TIMEOUT` (default: 60) frames after disappearing, so briefly covering a card does not clear it.

Equity Calculation

File: poker_hand/poker_main.py, library: treys

The equity method depends on available information:

Stage	Method	Speed
Preflop	Lookup table (169 canonical hand types)	Instant
Flop / Turn	Sample 200 random opponent pairs from remaining deck, evaluate with Treys	~50ms
River	Enumerate all possible opponent pairs, evaluate with Treys	~10ms

The Treys library represents cards as 32-bit integers with bit fields for rank, suit, and prime factorisation, enabling $O(1)$ hand evaluation via lookup rather than comparison.

SQLite Database

File: adaptive_learning/profile_store.py

All learning state is stored in a single SQLite file. The path is set via `PROFILES_DB_PATH` in `.env`, defaulting to `adaptive_learning/profiles.db`.

Key tables:

Table	Contents
players	Face embedding, first/last seen timestamps, session count, session notes (Claude debrief)
baselines	Mean and std dev for each physiological signal, baseline quality metadata
bandit_state	Alpha and beta dicts per context bucket, per player
personality_state	Bluff base rate, stress-bluff slope, HR-bluff slope, consistency score
showdowns	Per-hand record: prediction, actual result, peak signal stats, downsampled time series, Claude verdict

Nothing in the database identifies a real person. Players are referenced by a UUID generated at first recognition.